**MINISTERUL EDUCAȚIEI, CULTURII ȘI CERCETĂRII**
**AL REPUBLICII MOLDOVA**
**Universitatea Tehnică a Moldovei**
**Facultatea Calculatoare, Informatică și Microelectronică**
**Departamentul Inginerie Software și Automatică**

# Canter Daniel - FAF-242

# Report

Laboratory work n.1

**of Cryptography**

Checked by:

Zaica Maia, university assistant

DISA, FCIM, UTM

**Chișinău – 2026**

# 1. THEORY

## 1.1. The Caesar Cipher

In the Caesar cipher every letter of the message is replaced by the letter found $k$ positions further in the alphabet, wrapping around at the end. The letters are numbered from 0 to $n-1$ and the same key $k$ is used to encrypt and decrypt:

$$c = e_k(x) = (x + k) \bmod n, \quad m = d_k(y) = (y - k) \bmod n$$

The key is $k \in \{1, 2, ..., n-1\}$, since $k = 0$ leaves the text unchanged. For the Romanian alphabet $n = 31$, so there are only **30 keys**, and the cipher is broken by simply trying all of them (brute force).

## 1.2. The Caesar Cipher with a Permutation

A second key, a keyword $k_2$, is used to reorder the alphabet: first the letters of the keyword (each only once), then the rest of the letters in their normal order. The formulas stay the same, but $x$ and $y$ are now positions in the permuted alphabet.

The key space is every ordering of the alphabet times every shift:

$$31! \times 30 = 246\,685\,159\,625\,337\,684\,531\,766\,886\,400\,000\,000 \approx 2.47 \cdot 10^{35}$$

This is too many to brute force, but every letter is still always replaced by the same letter, so the cipher can be broken with frequency analysis.

## 2. IMPLEMENTATION

The lab is written in Python and split in three files:

- `common.py` holds the alphabet, text preparation, encryption, decryption and the input checks used by both tasks
- `caesar.py` is Task 1.1, the classic Caesar cipher
- `caesar_keyword.py` is Task 1.2, the Caesar cipher with a keyword

### 2.1. Letter Encoding

```
ALPHABET = "AĂÂBCDEFGHIÎJKLMNOPQRSȘTȚUVWXYZ"
N = len(ALPHABET)
```

The 31 letters are written in the order of Table 2, so the code of a letter is just its index in the string (`A` = 0, `Ă` = 1, …, `Z` = 30). The shift never uses ASCII or Unicode values.

### 2.2. Preparing the Text

```
def prepare(text):
    text = text.replace(" ", "").upper()
    invalid = [ch for ch in text if ch not in ALPHABET]
    if invalid:
        print(f"Invalid character(s): {' '.join(invalid)}. Allowed: only letters "
              f"of the Romanian alphabet {ALPHABET}, uppercase or lowercase.")
        return None
    return text
```

The spaces are removed and the text is converted to uppercase; Python's `upper()` also handles the Romanian letters (`ă` → `Ă`, `ș` → `Ș`). Every character that is not in the alphabet is collected and shown in the error message, and `None` is returned so the input is asked again. The keyword goes through the same function.

### 2.3. Encryption and Decryption

```
def encrypt(text, k, alphabet=ALPHABET):
    n = len(alphabet)
    return "".join(alphabet[(alphabet.index(ch) + k) % n] for ch in text)


def decrypt(text, k, alphabet=ALPHABET):
    n = len(alphabet)
    return "".join(alphabet[(alphabet.index(ch) - k) % n] for ch in text)
```

Each letter is turned into its position, shifted by $k$ and turned back into a letter, which is exactly $(x + k) \bmod n$ and $(y - k) \bmod n$. Python's `%` always returns a value between 0 and $n - 1$, so a negative $y - k$ wraps around without extra code. The alphabet is a parameter, so Task 1.2 uses the same two functions with the permuted alphabet.

### 2.4. Building the Permuted Alphabet

```
def permuted_alphabet(keyword, alphabet=ALPHABET):
    result = ""
    for ch in keyword + alphabet:
        if ch not in result:
            result += ch
    return result
```

The function walks over the keyword followed by the whole alphabet and keeps each letter only the first time it appears. The result always has all 31 letters, starting with the ones of the keyword.

### 2.5. Input Validation

```
def read_key(prompt):
    while True:
        raw = input(prompt).strip()
        if raw.isdecimal() and 1 <= int(raw) <= N - 1:
            return int(raw)
        print(f"Invalid key '{raw}'. The key must be an integer from 1 to {N - 1}.")


def read_keyword(prompt):
    while True:
        keyword = prepare(input(prompt))
        if keyword is None:
            continue
        if len(keyword) >= 7:
            return keyword
        print(f"Keyword too short ({len(keyword)} letters). "
              f"It must have at least 7 letters.")
```

Every input is asked again until it is valid. The key has to be a whole number from 1 to 30, and the keyword has to contain only Romanian letters and be at least 7 letters long.

### 2.6. Task 1.2 Program

```
def main():
    while True:
        op = read_choice("\nE - encrypt, D - decrypt: ", ["E", "D"])
        k = read_key(f"Key 1 (1-{N - 1}): ")
        keyword = read_keyword("Key 2 - keyword (at least 7 letters): ")
        alphabet = permuted_alphabet(keyword)
        print("Permuted alphabet:", " ".join(alphabet))
        if op == "E":
            print("Ciphertext:", encrypt(read_text("Message: "), k, alphabet))
        else:
            print("Message:", decrypt(read_text("Ciphertext: "), k, alphabet))
```

The program reads the operation and both keys, prints the permuted alphabet so it can be checked by hand, and then encrypts or decrypts the text. The Task 1.1 program is the same without the keyword.

# 3. RESULTS

## 3.1. Task 1.1 — Encryption

```
E - encrypt, D - decrypt: E
Key (1-30): 0
Invalid key '0'. The key must be an integer from 1 to 30.
Key (1-30): abc
Invalid key 'abc'. The key must be an integer from 1 to 30.
Key (1-30): 3
Message: cifrul cezar 2026!
Invalid character(s): 2 0 2 6 !. Allowed: only letters of the Romanian alphabet
AĂÂBCDEFGHIÎJKLMNOPQRSȘTȚUVWXYZ, uppercase or lowercase.
Message: cifrul cezar
Ciphertext: FKITXOFHÂBT
```

The wrong keys and the invalid characters are rejected and asked again. The last letter checks by hand: `Z` (30) + 3 = 33 mod 31 = 2, which is `Â`.

## 3.2. Task 1.1 — Decryption

```
E - encrypt, D - decrypt: D
Key (1-30): 3
Ciphertext: FKITXOFHÂBT
Message: CIFRULCEZAR
```

## 3.3. Task 1.2 — Encryption

```
E - encrypt, D - decrypt: E
Key 1 (1-30): 35
Invalid key '35'. The key must be an integer from 1 to 30.
Key 1 (1-30): 3
Key 2 - keyword (at least 7 letters): cripto
Keyword too short (6 letters). It must have at least 7 letters.
Key 2 - keyword (at least 7 letters): criptografie2
Invalid character(s): 2. Allowed: only letters of the Romanian alphabet
AĂÂBCDEFGHIÎJKLMNOPQRSȘTȚUVWXYZ, uppercase or lowercase.
Key 2 - keyword (at least 7 letters): criptografie
Permuted alphabet: C R I P T O G A F E Ă Â B D H Î J K L M N Q S Ș Ț U V W X Y Z
Message: Știința în țară
Ciphertext: VAOOȘWĂLȘWĂTD
```

`CRIPTOGRAFIE` gives the 10 distinct letters `C R I P T O G A F E` (the second `R` and `I` are dropped), followed by the other 21 letters in order. Checking one letter by hand: `Ș` is at position 23, and 23 + 3 = 26, which is `V`.

## 3.4. Task 1.2 — Decryption

```
E - encrypt, D - decrypt: D
Key 1 (1-30): 3
Key 2 - keyword (at least 7 letters): criptografie
Permuted alphabet: C R I P T O G A F E Ă Â B D H Î J K L M N Q S Ș Ț U V W X Y Z
Ciphertext: vaooșwălșwătd
Message: ȘTIINȚAÎNȚARĂ
```

## 4. CONCLUSION

The classic Caesar cipher over the Romanian alphabet has only 30 keys, so it can be broken instantly by trying all of them. Adding a keyword permutation raises the key space to $31! \times 30 \approx 2.47 \cdot 10^{35}$, which makes brute force impossible, but the cipher is still a simple substitution: each letter always becomes the same letter, so the letter frequencies of the language stay visible and frequency analysis breaks it.

The main takeaway is that a large key space alone does not make a cipher secure. From the implementation side, the letter codes have to come from the cipher's own alphabet and not from the character encoding, the modulo has to wrap negative numbers correctly, and all user input has to be checked.

## 5. SOURCE CODE

The full source code is available at `https://github.com/The1Dani/cs2026/tree/main/Lab1`.